

DESIGN AND ANALYSIS OF MACHINE LEARNING EXPERIMENTS

Guidelines for Machine Learning Experiments:

Designing machine learning (ML) experiments systematically is crucial for achieving reliable and reproducible results.

Systematic Machine Learning Experiment Design



Define Experiment Objective

Establish the problem statement, target variable, hypothesis, and success criteria.

Collect, clean, and transform data for the experiment.

Select and Prepare Dataset



Choose ML Models

Select appropriate machine learning models based on the problem type.

Divide data into training, validation, and test sets.

Split Data



Train and Optimize

Train the model and optimize hyperparameters.

Assess model performance using classification and regression metrics.

Evaluate Performance



Address Overfitting/Underfitting

Implement strategies to prevent overfitting and underfitting.

Use cross-validation and bootstrapping for model validation.

Perform Resampling



Deploy and Document

Deploy the model and ensure thorough documentation for reproducibility.

1. Define the Objective of the Experiment

Before starting an ML experiment, clearly define:

- **Problem Statement:** What problem are you trying to solve?
- **Target Variable:** What is the outcome variable (classification, regression, etc.)?
- **Hypothesis:** What do you expect to achieve? (e.g., "Algorithm A will outperform Algorithm B")
- **Success Criteria:** How will you measure success? (e.g., accuracy, precision, recall, RMSE, etc.)

Example: If you are building a spam detection model, your objective could be:

"To develop a classification model that accurately identifies spam emails with at least 95% accuracy and an F1-score above 0.90."

2. Select and Prepare the Dataset

Data is the foundation of any ML experiment, so choosing the right dataset is critical.

A. Collecting Data

- Use public datasets (e.g., Kaggle, UCI ML Repository) or collect your own data.
- Ensure data is representative of real-world scenarios.

B. Cleaning the Data

- Handle **missing values** (imputation, removal, or replacement).
- Remove **duplicates** to avoid bias.
- Fix **inconsistent data entries** (e.g., standardize date formats).
- Detect and handle **outliers** using techniques like Z-score or IQR.

C. Feature Engineering & Transformation

- Convert categorical variables into numerical (one-hot encoding, label encoding).
 - Normalize/standardize features if required (e.g., MinMaxScaler, StandardScaler).
-

3. Choose Machine Learning Models

Selecting the right model depends on the problem type:

- **Classification:** Logistic Regression, Decision Tree, Random Forest, SVM, Neural Networks, etc.
 - **Regression:** Linear Regression, Ridge, Lasso, Random Forest Regressor, etc.
 - **Clustering:** K-Means, Hierarchical Clustering, etc.
-

4. Split Data into Training, Validation, and Test Sets

To evaluate model performance properly:

- **Training Set (60-80%):** Used to train the model.
- **Validation Set (10-20%):** Used for hyperparameter tuning.
- **Test Set (10-20%):** Used for final model evaluation.

Alternative: Use **Cross-Validation (CV)** to get a better estimate of performance.

5. Train the Model and Optimize Hyperparameters

Training involves fitting the model to the training data.

- **Choose the right optimization algorithm** (e.g., Adam, SGD for deep learning).
-

6. Evaluate Model Performance

Use appropriate performance metrics based on the type of problem:

A. Classification Metrics

- **Accuracy:** Overall correctness of predictions.
- **Precision:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$ (relevant for imbalanced data).
- **Recall:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$ (important in medical diagnosis, fraud detection).

B. Regression Metrics

- **Mean Squared Error (MSE):** Measures squared differences (penalizes large errors).
- **Root Mean Squared Error (RMSE):** Square root of MSE, easier to interpret.
- **R² Score:** Measures how well the model explains variability in the data.

7. Avoid Overfitting and Underfitting

- **Overfitting:** Model performs well on training data but poorly on new data.
 - Solution: Use **regularization (L1, L2), dropout, pruning, or simpler models.**
- **Underfitting:** Model is too simple and fails to capture patterns.
 - Solution: Use **more complex models, feature engineering, or more data.**

8. Perform Resampling and Cross-Validation

- **K-Fold Cross-Validation:** Splits data into K subsets and trains the model K times, each time using a different subset for testing.
- **Bootstrapping:** Creates multiple training samples by resampling with replacement, useful for estimating model confidence intervals.

9. Compare Different Models and Perform Statistical Tests

- Run experiments on multiple algorithms and compare results.

- Use statistical tests like **t-test**, **McNemar's test** to check for significant differences between models.
-

10. Deploy the Model

- Convert the trained model into an API using **Flask**.
 - Monitor model performance in production.
 - Continuously update the model if necessary.
-

11. Document and Reproduce the Experiment

Ensure your experiment can be reproduced by others:

- **Record experiment settings** (dataset versions, model parameters, preprocessing steps).
- **Use notebooks (Jupyter, Colab)**
- **Store model weights, logs, and performance metrics** for future reference.

Cross Validation (CV) and Resampling – K-Fold CV:

Cross-validation and resampling are essential techniques in machine learning to evaluate model performance, prevent overfitting, and ensure model generalization. These techniques help assess how well a model will perform on unseen data.

What is Cross-Validation (CV)?

Cross-validation is a technique used to assess the performance of a machine learning model by dividing the dataset into multiple subsets and training/testing the model on different splits of the data.

Why Use Cross-Validation?

- ✓ **Prevents Overfitting** – Ensures that the model generalizes well to new data.
 - ✓ **Better Performance Estimation** – Provides a more accurate estimate of model performance than a single train-test split.
-

K-Fold Cross-Validation

K-Fold Cross-Validation is one of the most widely used CV techniques. It involves dividing the dataset into **K equal-sized folds** (subsets), where:

- The model is trained on $K-1$ folds and tested on the remaining fold.
- The process repeats K times, each time using a different fold for testing.
- The final performance score is the average of all K iterations.

Steps in K-Fold Cross-Validation

1. Shuffle the dataset to ensure randomness.
2. Split it into **K equal parts (folds)**.
3. For each fold:
 - Train the model on $K-1$ folds.
 - Test it on the remaining fold.
4. Compute the performance metric (e.g., accuracy, RMSE) for each fold.
5. Take the **average of all K runs** as the final model performance.

Choosing K in K-Fold CV:

- **K = 5 or 10** is commonly used (5-Fold CV or 10-Fold CV).
- A smaller K (e.g., K=3) increases bias (less reliable estimate).
- A larger K (e.g., K=20) increases variance (more computational cost).

Example: 5-Fold Cross-Validation

Suppose we have a dataset with 1000 samples and use **5-Fold CV**:

- **Fold 1:** Train on 800 samples, test on 200 samples.
- **Fold 2:** Train on 800 samples, test on different 200 samples.
- **Fold 3:** Train on 800 samples, test on another 200 samples.

- **Fold 4:** Train on 800 samples, test on another 200 samples.
- **Fold 5:** Train on 800 samples, test on the final 200 samples.

✓ **Final Performance Score** = Average of the accuracy (or other metric) across all 5 folds.

Advantages of K-Fold CV:

- Utilizes all data for training and testing.
 - Reduces bias in model performance estimation.
 - Helps detect overfitting issues.
-

Bootstrapping

Bootstrapping is a resampling technique used to estimate the accuracy of machine learning models by creating multiple training datasets from the original data **with replacement**.

How Bootstrapping Works:

1. Randomly sample the dataset **with replacement** to create a new dataset of the same size.
2. Train the model on this new sample.
3. Test the model on the data points **not included** in the training set (called "out-of-bag" (OOB) samples).
4. Repeat the process multiple times (e.g., 1000 times).
5. Compute the average performance metric across all iterations.

Key Concept: Sampling with Replacement

- Each bootstrap sample may contain duplicate entries because each selection is independent.
- Some data points **may not be included** in a given bootstrap sample.
- Approximately **63% of the original data** appears in each bootstrap sample, while the remaining **37% serve as test data**.

Bootstrapping: Small Calculation Example

Let's take a small dataset and manually perform bootstrapping.

Step 1: Consider a Small Dataset

Assume we have a dataset with 5 values:

$$D = \{2, 4, 6, 8, 10\}$$

Step 2: Generate Bootstrap Samples

We randomly sample **with replacement**. Let's generate **three bootstrap samples**:

Sample #	Bootstrap Sample
1	{4, 6, 6, 8, 10}
2	{2, 2, 4, 8, 10}
3	{2, 4, 4, 6, 8}

Step 3: Compute Sample Mean for Each Bootstrap Sample

Now, let's calculate the **mean** for each bootstrap sample:

- Sample 1 Mean:

$$\frac{(4 + 6 + 6 + 8 + 10)}{5} = \frac{34}{5} = 6.8$$

- Sample 2 Mean:

$$\frac{(2 + 2 + 4 + 8 + 10)}{5} = \frac{26}{5} = 5.2$$

- Sample 3 Mean:

$$\frac{(2 + 4 + 4 + 6 + 8)}{5} = \frac{24}{5} = 4.8$$

Step 4: Compute Overall Estimate

The bootstrap estimate of the mean is the average of all bootstrap sample means:

$$\frac{(6.8 + 5.2 + 4.8)}{3} = \frac{16.8}{3} = 5.6$$

Step 5: Compute Bootstrap Confidence Interval (Approximate)

To estimate a **95% confidence interval**, we sort the bootstrap means:

$$\{4.8, 5.2, 6.8\}$$

For a small sample size, the **2.5th percentile** is the lowest value (4.8), and the **97.5th percentile** is the highest value (6.8).

Thus, the **95% confidence interval** for the mean is **(4.8, 6.8)**.

Final Results

- **Original Mean (Actual Dataset Mean) =**

$$\frac{(2 + 4 + 6 + 8 + 10)}{5} = \frac{30}{5} = 6$$

- **Bootstrap Estimated Mean = 5.6** (slightly different due to sampling variability)
- **95% Confidence Interval = (4.8, 6.8)**

Advantages of Bootstrapping:

- ✓ Works well with **small datasets** (unlike K-Fold CV, which requires more data).
- ✓ Generates **confidence intervals** for performance metrics.
- ✓ Useful in **ensemble methods** like Bagging (e.g., Random Forest)

measuring classifier performance assessing a single classification algorithm

The **confusion matrix** is a fundamental tool for evaluating the performance of a classification model. It provides insight into how well a model distinguishes between classes by breaking down the predictions into four categories.

1. Understanding the Confusion Matrix

For a binary classification problem (e.g., detecting spam emails), the confusion matrix looks like this:

Actual / Predicted	Predicted Positive	Predicted Negative
Actual Positive (P)	True Positive (TP)	False Negative (FN)
Actual Negative (N)	False Positive (FP)	True Negative (TN)

Terminology:

- **True Positive (TP):** Model correctly predicts a positive case.
 - **False Positive (FP):** Model incorrectly predicts a positive case when it's actually negative (Type I Error).
 - **False Negative (FN):** Model incorrectly predicts a negative case when it's actually positive (Type II Error).
 - **True Negative (TN):** Model correctly predicts a negative case.
-

2. Example Confusion Matrix

Let's assume we have a dataset where a model is classifying whether an email is spam (**Positive**) or not spam (**Negative**).

Actual / Predicted	Spam (Positive)	Not Spam (Negative)
Spam (Actual Positive)	50 (TP)	10 (FN)
Not Spam (Actual Negative)	5 (FP)	100 (TN)

Here:

- **TP = 50** (Correctly identified spam emails)
 - **FP = 5** (Non-spam emails incorrectly classified as spam)
 - **FN = 10** (Spam emails missed by the model)
 - **TN = 100** (Correctly identified non-spam emails)
-

3. Calculating Evaluation Metrics

(A) Accuracy

Measures the overall correctness of the model.

$$\begin{aligned}\text{Accuracy} &= \frac{TP + TN}{TP + FP + FN + TN} \\ &= \frac{50 + 100}{50 + 5 + 10 + 100} = \frac{150}{165} = 0.909 \\ \text{Accuracy} &= 90.9\%\end{aligned}$$

(B) Precision (Positive Predictive Value)

Measures how many of the predicted positive cases were actually positive.

$$\begin{aligned}\text{Precision} &= \frac{TP}{TP + FP} \\ &= \frac{50}{50 + 5} = \frac{50}{55} = 0.909 \\ \text{Precision} &= 90.9\%\end{aligned}$$

(C) Recall (Sensitivity, True Positive Rate)

Measures how many of the actual positive cases were correctly identified.

$$\begin{aligned}\text{Recall} &= \frac{TP}{TP + FN} \\ &= \frac{50}{50 + 10} = \frac{50}{60} = 0.833 \\ \text{Recall} &= 83.3\%\end{aligned}$$

(E) False Positive Rate (FPR)

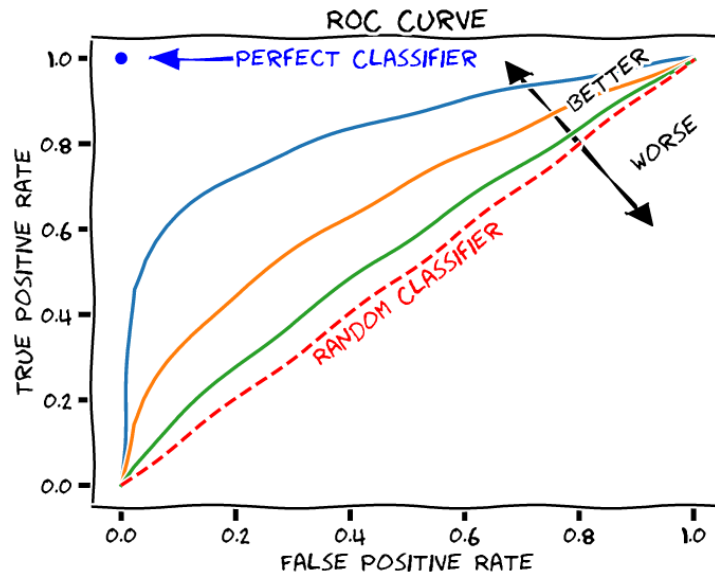
Measures how often negative cases are incorrectly classified as positive.

$$\begin{aligned}\text{FPR} &= \frac{FP}{FP + TN} \\ &= \frac{5}{5 + 100} = \frac{5}{105} = 0.048 \\ \text{FPR} &= 4.8\%\end{aligned}$$

(F) ROC Curve and AUC

The ROC Curve plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different threshold values. The AUC (Area Under the Curve) represents the overall ability of the model to distinguish between classes.

A higher AUC (closer to 1.0) indicates better classification performance.



4. Interpretation of the Results

- **Accuracy (90.9%)** → High, meaning the model performs well overall.
- **Precision (90.9%)** → The model is very good at predicting spam without many false positives.
- **Recall (83.3%)** → Some spam emails were missed, meaning false negatives are a concern.
- **F1-Score (87.0%)** → A good balance between precision and recall.
- **False Positive Rate (4.8%)** → Very few non-spam emails were mistakenly classified as spam.

Comparing two classification algorithms

Paired t-test with K-Fold Cross-Validation

The **paired t-test** is used to compare the performance of **two classification algorithms** across multiple folds in **K-Fold Cross-Validation**. It helps determine whether the performance difference is **statistically significant**.

1. Why Use a Paired t-test in K-Fold CV?

When comparing two classifiers (e.g., **Model A vs. Model B**), performance variations can arise due to:

- Different data splits
- Random initialization
- Training variations

A **paired t-test** checks if the observed difference is **consistent** across different K-Folds or if it is just due to chance.

2. Steps for Applying Paired t-test in K-Fold CV

Step 1: Perform K-Fold Cross-Validation

- Train both classifiers on the same **K-fold splits**.
- Record their performance (e.g., **accuracy, precision, recall**) on each fold.

Step 2: Calculate the Performance Differences

- Compute the difference in performance between the two classifiers for each fold.

Step 3: Compute the Mean and Standard Deviation of Differences

- Calculate the **mean difference** and **standard deviation** of the differences.

Step 4: Compute the t-statistic

The paired t-test statistic is given by:

$$t = \frac{\bar{d}}{s_d / \sqrt{K}}$$

where:

- $\bar{d}$ = Mean of the performance differences
- s_d = Standard deviation of the performance differences
- K = Number of folds

Step 5: Compare with t-critical value

- Check the **t-table** for a significance level (e.g., $\alpha = 0.05$).

- If t is greater than the critical value, the difference is **statistically significant**.

Critical values of t for one-tailed tests

Significance level (α)

Degrees of freedom (df)	.2	.15	.1	.05	.025	.01	.005	.001
1	1.376	1.963	3.078	6.314	12.706	31.821	63.657	318.309
2	1.061	1.386	1.886	2.920	4.303	6.965	9.925	22.327
3	0.978	1.250	1.638	2.353	3.182	4.541	5.841	10.215
4	0.941	1.190	1.533	2.132	2.776	3.747	4.604	7.173
5	0.920	1.156	1.476	2.015	2.571	3.365	4.032	5.893
6	0.906	1.134	1.440	1.943	2.447	3.143	3.707	5.208
7	0.896	1.119	1.415	1.895	2.365	2.998	3.499	4.785
8	0.889	1.108	1.397	1.860	2.306	2.896	3.355	4.501
9	0.883	1.100	1.383	1.833	2.262	2.821	3.250	4.297
10	0.879	1.093	1.372	1.812	2.228	2.764	3.169	4.144
11	0.876	1.088	1.363	1.796	2.201	2.718	3.106	4.025
12	0.873	1.083	1.356	1.782	2.179	2.681	3.055	3.930
13	0.870	1.079	1.350	1.771	2.160	2.650	3.012	3.852
14	0.868	1.076	1.345	1.761	2.145	2.624	2.977	3.787
15	0.866	1.074	1.341	1.753	2.131	2.602	2.947	3.733
16	0.865	1.071	1.337	1.746	2.120	2.583	2.921	3.686
17	0.863	1.069	1.333	1.740	2.110	2.567	2.898	3.646
18	0.862	1.067	1.330	1.734	2.101	2.552	2.878	3.610
19	0.861	1.066	1.328	1.729	2.093	2.539	2.861	3.579
20	0.860	1.064	1.325	1.725	2.086	2.528	2.845	3.552
21	0.859	1.063	1.323	1.721	2.080	2.518	2.831	3.527
22	0.858	1.061	1.321	1.717	2.074	2.508	2.819	3.505
23	0.858	1.060	1.319	1.714	2.069	2.500	2.807	3.485
24	0.857	1.059	1.318	1.711	2.064	2.492	2.797	3.467
25	0.856	1.058	1.316	1.708	2.060	2.485	2.787	3.450
26	0.856	1.058	1.315	1.706	2.056	2.479	2.779	3.435
27	0.855	1.057	1.314	1.703	2.052	2.473	2.771	3.421
28	0.855	1.056	1.313	1.701	2.048	2.467	2.763	3.408
29	0.854	1.055	1.311	1.699	2.045	2.462	2.756	3.396
30	0.854	1.055	1.310	1.697	2.042	2.457	2.750	3.385
40	0.851	1.050	1.303	1.684	2.021	2.423	2.704	3.307
50	0.849	1.047	1.299	1.676	2.009	2.403	2.678	3.261
60	0.848	1.045	1.296	1.671	2.000	2.390	2.660	3.232
70	0.847	1.044	1.294	1.667	1.994	2.381	2.648	3.211
80	0.846	1.043	1.292	1.664	1.990	2.374	2.639	3.195
100	0.845	1.042	1.290	1.660	1.984	2.364	2.626	3.174
1000	0.842	1.037	1.282	1.646	1.962	2.330	2.581	3.098
Infinite	0.842	1.036	1.282	1.645	1.960	2.326	2.576	3.090

3. Example: Paired t-test with K = 5

Suppose we compare **Decision Tree** vs. **Random Forest** using **5-Fold CV**.

Fold	Decision Tree Accuracy (%)	Random Forest Accuracy (%)	Difference dd
1	78	82	-4
2	80	84	-4
3	76	81	-5
4	82	85	-3
5	79	83	-4

Step 1: Compute Mean Difference

$$\bar{d} = \frac{-4 + (-4) + (-5) + (-3) + (-4)}{5} = \frac{-20}{5} = -4$$

Step 2: Compute Standard Deviation of Differences

1. Compute each squared deviation from the mean:

$$(-4 + 4)^2 = 0, \quad (-4 + 4)^2 = 0, \quad (-5 + 4)^2 = 1, \quad (-3 + 4)^2 = 1, \quad (-4 + 4)^2 = 0$$

2. Compute variance:

$$s_d^2 = \frac{(0 + 0 + 1 + 1 + 0)}{5 - 1} = \frac{2}{4} = 0.5$$

3. Compute standard deviation:

$$s_d = \sqrt{0.5} = 0.707$$

Step 3: Compute t-value

$$t = \frac{-4}{0.707/\sqrt{5}}$$
$$t = \frac{-4}{0.316} = -12.66$$

Step 4: Compare with Critical Value

For **K = 5** folds and **$\alpha = 0.05$** , the **critical t-value (two-tailed, df = 4)** is **2.776**.

Since **$|t| = 12.66 > 2.776$** , we **reject the null hypothesis**. This means the **performance difference is statistically significant**, and **Random Forest performs**

significantly better than Decision Tree.

- **Paired t-test** determines if one classifier **performs significantly better** than another.
- Works with **K-Fold CV** by comparing performance differences across folds.
- If **t-value > critical value**, we reject the null hypothesis (i.e., performance difference is not due to randomness).

McNemar's Test:

McNemar's test is a **statistical test** used to compare the performance of two classification algorithms **on the same dataset**. It checks if there is a **significant difference in the misclassification patterns** between the two models.

It is particularly useful when the same samples are used for both classifiers, making it ideal for **paired comparisons** (dependent samples).

Hypotheses in McNemar's Test

- **Null Hypothesis (H_0)**: Both classifiers make errors in the **same way** (i.e., there is no significant difference).
- **Alternative Hypothesis (H_A)**: The classifiers make errors **differently**, meaning one model performs significantly better than the other.

1. Constructing the Contingency Table

McNemar's test uses a **2x2 contingency table** based on the classification results of two models:

	Model 2 Correct (B)	Model 2 Incorrect (B)	Row Total
Model 1 Correct (A)	n_{11}	n_{10}	$n_{1\cdot}$
Model 1 Incorrect (A)	n_{01}	n_{00}	$n_{0\cdot}$
Column Total	$n_{\cdot 1}$	$n_{\cdot 0}$	N

- n_{11} = Number of samples correctly classified by **both** models.
- n_{00} = Number of samples misclassified by **both** models.
- n_{10} = Samples **correct** in Model 1 but **incorrect** in Model 2.
- n_{01} = Samples **incorrect** in Model 1 but **correct** in Model 2.

McNemar's test only considers n_{10} and n_{01} since they represent disagreements.

2. McNemar's Test Statistic

The McNemar test statistic is calculated using:

$$\chi^2 = \frac{(n_{10} - n_{01})^2}{n_{10} + n_{01}}$$

This follows a **chi-square distribution with 1 degree of freedom**.

- If χ^2 is **greater** than the critical value from the chi-square table (at $\alpha = 0.05$, $df = 1 \rightarrow$ critical value = **3.841**), we **reject H_0** and conclude there is a significant difference.

3. Example Calculation

Suppose we tested **100 samples** using **Decision Tree (Model 1)** and **Random Forest (Model 2)**.

The **confusion matrix** (contingency table) is:

	Random Forest Correct	Random Forest Incorrect	Row Total
Decision Tree Correct	60	20	80
Decision Tree Incorrect	10	10	20
Column Total	70	30	100

From this table:

- $n_{10} = 20$ (Correct in Decision Tree, Incorrect in Random Forest)
- $n_{01} = 10$ (Incorrect in Decision Tree, Correct in Random Forest)

$$\chi^2 = \frac{(20 - 10)^2}{20 + 10} = \frac{100}{30} = 3.33$$

Degrees of freedom (df)	Significance level (α)							
	.99	.975	.95	.9	.1	.05	.025	.01
1	-----	0.001	0.004	0.016	2.706	3.841	5.024	6.635
2	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210
3	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345
4	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277
5	0.554	0.831	1.145	1.610	9.236	11.070	12.833	15.086
6	0.872	1.237	1.635	2.204	10.645	12.592	14.449	16.812
7	1.239	1.690	2.167	2.833	12.017	14.067	16.013	18.475
8	1.646	2.180	2.733	3.490	13.362	15.507	17.535	20.090
9	2.088	2.700	3.325	4.168	14.684	16.919	19.023	21.666
10	2.558	3.247	3.940	4.865	15.987	18.307	20.483	23.209
11	3.053	3.816	4.575	5.578	17.275	19.675	21.920	24.725
12	3.571	4.404	5.226	6.304	18.549	21.026	23.337	26.217
13	4.107	5.009	5.892	7.042	19.812	22.362	24.736	27.688
14	4.660	5.629	6.571	7.790	21.064	23.685	26.119	29.141
15	5.229	6.262	7.261	8.547	22.307	24.996	27.488	30.578
16	5.812	6.908	7.962	9.312	23.542	26.296	28.845	32.000
17	6.408	7.564	8.672	10.085	24.769	27.587	30.191	33.409
18	7.015	8.231	9.390	10.865	25.989	28.869	31.526	34.805
19	7.633	8.907	10.117	11.651	27.204	30.144	32.852	36.191
20	8.260	9.591	10.851	12.443	28.412	31.410	34.170	37.566
21	8.897	10.283	11.591	13.240	29.615	32.671	35.479	38.932
22	9.542	10.982	12.338	14.041	30.813	33.924	36.781	40.289
23	10.196	11.689	13.091	14.848	32.007	35.172	38.076	41.638
24	10.856	12.401	13.848	15.659	33.196	36.415	39.364	42.980
25	11.524	13.120	14.611	16.473	34.382	37.652	40.646	44.314
26	12.198	13.844	15.379	17.292	35.563	38.885	41.923	45.642
27	12.879	14.573	16.151	18.114	36.741	40.113	43.195	46.963
28	13.565	15.308	16.928	18.939	37.916	41.337	44.461	48.278
29	14.256	16.047	17.708	19.768	39.087	42.557	45.722	49.588
30	14.953	16.791	18.493	20.599	40.256	43.773	46.979	50.892
40	22.164	24.433	26.509	29.051	51.805	55.758	59.342	63.691
50	29.707	32.357	34.764	37.689	63.167	67.505	71.420	76.154
60	37.485	40.482	43.188	46.459	74.397	79.082	83.298	88.379
70	45.442	48.758	51.739	55.329	85.527	90.531	95.023	100.425
80	53.540	57.153	60.391	64.278	96.578	101.879	106.629	112.329
100	61.754	65.647	69.126	73.291	107.565	113.145	118.136	124.116
1000	70.065	74.222	77.929	82.358	118.498	124.342	129.561	135.807

4. Compare with Critical Value

- Chi-square critical value at $df = 1, \alpha = 0.05 = 3.841$
- Our computed $\chi^2 = 3.33$

Since $3.33 < 3.841$, we fail to reject the null hypothesis.

This means **there is no significant difference** between Decision Tree and Random Forest.

5. Interpretation

- If $\chi^2 > 3.841$, we reject H_0 , meaning **one model is significantly better.**
- If $\chi^2 < 3.841$, we fail to reject H_0 , meaning **the difference is not statistically significant.**